1. Product Backlog

We have identified the following features needed for the system. These make up the Product Backlog:

- Set up scenario, area, and resource data
- Calculate travel time and run a baseline allocation
- Run an optimized allocation and compare it with the baseline

These are prioritized in this order because the optimization needs the travel data, and the travel data needs the scenario data to already exist.

2. Sprint 1 — Setup and Scenario Data

Story A: Add areas and resources

Story: As a coordinator, I want to enter affected areas and available resources, so that the system has the data needed to plan an allocation.

Tasks

- Setup backend and database.
- Create tables for scenarios, areas, and resources.
- Build a form to add area details (population, severity, location).
- Build a form to add resource base details (location, quantity).

Acceptance Test Criteria

- Check whether an area can be added with population and severity.
- Check whether a resource base can be added with quantity.
- Check whether the data is saved correctly in the database.

Story B: View the scenario on a map

Story: As a coordinator, I want to see all areas and bases on a map, so that I can understand the situation before running an allocation.

Tasks

- Add a React dashboard.
- Add a Leaflet map.
- Plot areas by severity and bases as markers.

Acceptance Test Criteria

- Check whether all added areas show up on the map with correct severity.
- Check whether all resource bases show up as markers.

3. Sprint 2 — Travel Time and Baseline Allocation

Story A: Calculate travel time

Story: As a coordinator, I want the system to calculate travel time between bases and areas, so that distance is considered in the allocation.

Tasks

- Connect to OSRM.
- Build the travel-time matrix and store it.

Acceptance Test Criteria

- Check whether travel time is calculated for every base-area pair.
- Check whether it is stored correctly.

Story B: Run a baseline allocation

Story: As a coordinator, I want a simple nearest-base allocation, so that I have something to compare the optimized result against.

Tasks

- Build the greedy/nearest-base logic.
- Store the result in the allocations table.
- Show baseline result and basic metrics on the dashboard.

Acceptance Test Criteria

- Check whether each area gets resources from the nearest base with availability.
- Check whether a base never sends more than it has.
- Check whether response time and unmet demand are shown after the run.

4. Sprint 3 — Optimization and Comparison

Story A: Run the optimized allocation

Story: As a coordinator, I want an optimized allocation, so that resources are used more efficiently than the simple method.

Tasks

- Build the optimization model using OR-Tools/PuLP.
- Add constraints for base capacity and area demand.
- Store the result separately from the baseline.

Acceptance Test Criteria

- Check whether the solver respects capacity and demand limits.
- Check whether the optimized result is stored correctly.

Story B: Compare baseline and optimized results

Story: As a coordinator, I want to compare the two results side by side, so that I can see which one performs better.

Tasks

- Calculate comparison metrics.
- Build a side-by-side view on the dashboard.

Acceptance Test Criteria

- Check whether both results can be viewed together.
- Check whether the metrics correctly show the difference.

Story C: Test multiple scenarios

Story: As a coordinator, I want to test the system on a few sample scenarios, so that I know it works beyond one case.

Tasks

- Create 2–3 sample scenarios.
- Run both methods on each and record results.

Acceptance Test Criteria

- Check whether each scenario runs without errors.
- Check whether results are recorded for all of them.